P.D.E.A's
Prof. Ramkrishna More A.C.S. College (Autonomous) Akurdi,Pune-44
Department Of Computer Science
T.Y.B.Sc(CS)
CSMAP-354 Operating System and Data Science Laboratory

Assingment-1: The Data Science Environment

Qu. 1 What is Data Set in Python?

A Dataset is a set of data grouped into a collection with which developers can work to meet their goals. In a dataset, the rows represent the number of data points and the columns represent the features of the Dataset. They are mostly used in fields like machine learning, business, and government to gain insights, make informed decisions, or train algorithms. Datasets may vary in size and complexity and they mostly require cleaning and preprocessing to ensure data quality and suitability for analysis or modeling.

Qu. 2 What is Data Frame in Python?

Ans : A Data Frame in Python, primarily found within the Pandas library, is a two-dimensional, mutable, tabular data structure with labeled axes (rows and columns). It is analogous to a spreadsheet or a SQL table.

Qu . 3 How to create Data Frame?

Here are common ways to create a Data Frame:

1. **From a Dictionary of Lists/Arrays:** This is a widely used method where keys represent column names and values are lists or NumPy arrays containing the data for each column.

```
import pandas as pd
data = {'Name': ['Ram', 'Laxman', 'Sita'],
        'Age': [35, 32, 30],
        'City': ['Ayodhya', 'Varanasi', 'Banaras']}
df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name  Age      City
0    Ram   35   Ayodhya
1 Laxman   32  Varanasi
2   Sita   30   Banaras
```

2. **From a List of Lists:** Each inner list represents a row, and you can optionally provide column names.

```
import pandas as pd
data = [['Ram', 35, 'Pune'],
        ['Sham', 32, 'Mumbai'],
        ['Gita', 30, 'Nagpur']]
df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
print(df)
```

3. **From a List of Dictionaries:** Each dictionary in the list represents a row, with keys as column names and values as cell data.

```
import pandas as pd
data = [{'Name': 'Ram', 'Age': 25, 'City': 'Nagpur'},
        {'Name': 'Sham', 'Age': 30, 'City': 'Mumbai'},
        {'Name': 'Gita', 'Age': 35, 'City': 'Pune'}]
df = pd.DataFrame(data)
print(df)
```

4. **From a NumPy Array:** You can convert a NumPy array directly into a DataFrame, optionally specifying column names and an index.

```
import pandas as pd
import numpy as np
data = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])
df = pd.DataFrame(data, columns=['Col-A', 'Col-B', 'Col-C'])
print(df)
```

5. **Creating an Empty DataFrame:** You can create an empty DataFrame and then add data to it.

```
import pandas as pd
df = pd.DataFrame()
print(df)

# Add a column
df['Name'] = ['Ram', 'Sham']
print(df)
```

Assingment-1: The Data Science Environment

Set-A

1. **Write a Python program to create a dataframe containing columns name, age and percentage. Add 10 rows to the dataframe. View the dataframe.**

```
import pandas as pd
data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
df = pd.DataFrame(data)
print(df)
```

2. **Write a Python program to print the shape, number of rows-columns, data types, feature names and the description of the data**

```
import pandas as pd
data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
df = pd.DataFrame(data)
print("Shape of the DataFrame:", df.shape)
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
print("\nData types of each column:")
print(df.dtypes)
print("\nFeature names (column names):")
print(df.columns.tolist())
print("\nDescription of the DataFrame:")
print(df.describe(include='all'))
```

3. **Write a Python program to view basic statistical details of the data.**

```
import pandas as pd
data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
```

```
df = pd.DataFrame(data)

print("DataFrame:")
print(df)
print("\nBasic statistical details of the data:")
print(df.describe())
```

4. **Write a Python program to Add 5 rows with duplicate values and missing values. Add a column 'remarks' with empty values. Display the data**

```
import pandas as pd
data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
df = pd.DataFrame(data)
add_data = {
    'name': ['K', 'L', None, 'N', 'O'],
    'age': [23, 30, 25, None, 29],
    'percentage': [85.5, None, 78.6, 79.5, 91.2]
}
add_df = pd.DataFrame(add_data)
df = pd.concat([df, add_df], ignore_index=True)
df['remarks'] = ""
print(df)
```

5. **Write a Python program to get the number of observations, missing values and duplicate values.**

```
import pandas as pd
data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
df = pd.DataFrame(data)
add_data = {
    'name': ['K', 'L', None, 'N', 'O'],
    'age': [23, 30, 25, None, 29],
    'percentage': [85.5, None, 78.6, 79.5, 91.2]
}
add_df = pd.DataFrame(add_data)
df = pd.concat([df, add_df], ignore_index=True)
```

```
df['remarks'] = ""
print(df)
num_obser = df.shape[0]
num_missing_values = df.isnull().sum().sum()
num_duplicate_rows = df.duplicated().sum()

print(f"Number of observations: {num_obser}")
print(f"Number of missing values: {num_missing_values}")
print(f"Number of duplicate rows: {num_duplicate_rows}")
```

6. **Write a Python program to drop 'remarks' column from the dataframe. Also drop all null and empty values. Print the modified data.**

```
import pandas as pd

data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
df = pd.DataFrame(data)
add_data = {
    'name': ['K', 'L', None, 'N', 'O'],
    'age': [23, 30, 25, None, 29],
    'percentage': [85.5, None, 78.6, 79.5, 91.2]
}
add_df = pd.DataFrame(add_data)
df = pd.concat([df, add_df], ignore_index=True)
df['remarks'] = ""
print(df)
df = df.drop(columns=['remarks'])
df = df.dropna()
print( )
print(df)
```

7. **Write a Python program to generate a line plot of name vs percentage**

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
```

```
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}
df = pd.DataFrame(data)
add_data = {
    'name': ['K', 'L', None, 'N', 'O'],
    'age': [23, 30, 25, None, 29],
    'percentage': [85.5, None, 78.6, 79.5, 91.2]
}
add_df = pd.DataFrame(add_data)
df = pd.concat([df, add_df], ignore_index=True)
df['remarks'] = ""
print(df)
df = df.drop(columns=['remarks'])
df = df.dropna()
plt.figure(figsize=(10, 6))
plt.plot(df['name'], df['percentage'], marker='o', linestyle='-')
plt.xlabel('Name')
plt.ylabel('Percentage')
plt.title('Name vs Percentage')
plt.grid(True)
plt.tight_layout()
plt.show()
```

8. **Write a Python program to generate a scatter plot of name vs percentage**

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J'],
    'age': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'percentage': [85.5, 92.3, 78.6, 88.4, 79.5, 91.2, 85.7, 90.1, 87.6, 83.4]
}

df = pd.DataFrame(data)

add_data = {
    'name': ['K', 'L', None, 'N', 'O'],
    'age': [23, 30, 25, None, 29],
    'percentage': [85.5, None, 78.6, 79.5, 91.2]
}
add_df = pd.DataFrame(add_data)
df = pd.concat([df, add_df], ignore_index=True)
```

```
df['remarks'] = ""
print(df)
df = df.drop(columns=['remarks'])
df = df.dropna()

plt.figure(figsize=(10, 6))
plt.scatter(df['name'], df['percentage'], c='blue', marker='o')
plt.xlabel('Name')
plt.ylabel('Percentage')
plt.title('Name vs Percentage')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Set-B

1. **Download the heights and weights dataset and load the dataset from a given csv file into a dataframe.Print the first, last 10 rows and random 20 rows. (https://www.kaggle.com/burnoutminer/heightsand-weights-dataset)**

```
import pandas as pd

# Load the dataset
url = _____
df = pd.read_csv(url)

# Print the shape of the DataFrame
print("Shape of the DataFrame:")
print(df.shape)

# Print the size of the DataFrame
print("\nSize of the DataFrame:")
print(df.size)

# Print the datatypes of the DataFrame
print("\nDatatypes of the DataFrame:")
print(df.dtypes)

# Print the first 10 rows
print("\nFirst 10 rows:")
```

```
print(df.head(10))

# Print the last 10 rows
print("\nLast 10 rows:")
print(df.tail(10))

# Print 20 random rows
print("\nRandom 20 rows:")
print(df.sample(20))
```

2. Write a Python program to find the shape, size, datatypes of the dataframe object.

```
import pandas as pd

# Load the dataset
file_path = _____
df = pd.read_csv(file_path)

# Print the shape of the DataFrame
print("Shape of the DataFrame:")
print(df.shape)

# Print the size of the DataFrame
print("\nSize of the DataFrame:")
print(df.size)

# Print the datatypes of the DataFrame
print("\nDatatypes of the DataFrame:")
```

```
print(df.dtypes)
```

3. **Write a Python program to view basic statistical details of the data.**

```
import pandas as pd

# Load the dataset
file_path =_______
df = pd.read_csv(file_path)

# Print basic statistical details of the data
print("Basic statistical details of the data:")
print(df.describe())
```

4. **Write a Python program to get the number of observations, missing values and nan values.**

```
import pandas as pd

# Load the dataset
file_path =
df = pd.read_csv(file_path)

# Get the number of observations
num_observations = df.shape[0]
print(f"Number of observations: {num_observations}")

# Get the number of missing values per column
missing_values = df.isnull().sum()
print("\nNumber of missing values per column:")
print(missing_values)

# Get the number of NaN values per column
nan_values = df.isna().sum()
print("\nNumber of NaN values per column:")
print(nan_values)
```

5. **Write a Python program to generate a scatter plot of height vs weight**

```
import matplotlib.pyplot as plt
```

```
# Sample data
heights = [150, 160, 170, 180, 190]
weights = [50, 60, 70, 80, 90]

# Create scatter plot
plt.scatter(heights, weights, color='blue', marker='o')

# Add labels and title
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.title('Height vs Weight Scatter Plot')

# Display the plot
plt.grid(True)
plt.show()
```

Self Study:

6. **Write a Python program to add a column to the dataframe "BMI" which is calculated as : weight/height2**
7. **Write a Python program to find the maximum and minimum BMI.**